

AKSA-SDS Summer Internship

Name: Abdur Rehman Tariq

Dept: AI

What is RAG

Retrieval Augmented Generation, commonly known as RAG, is a technique that combines a large language model with an external knowledge source. Instead of relying only on what the model learned during training, the system first retrieves relevant information from a document collection and then passes that information to the language model along with the user question. The model uses this retrieved content to produce an answer that is grounded in the actual source material rather than in the model's internal parameters alone.

This approach separates the reasoning ability of the language model from the storage of factual knowledge. The knowledge base can be updated at any time by adding, removing, or editing documents, and the language model does not need to be retrained for the system to reflect that change.

Why RAG is Needed

Standalone large language models are trained on a fixed dataset up to a certain point in time. This creates several limitations that RAG is designed to address.

- Knowledge cutoff: a standalone model has no awareness of information that appeared after its training data was collected.
- Hallucination: when a model does not know an answer, it may generate a response that sounds plausible but is factually incorrect.
- No access to private or domain specific data: a general purpose model has not seen an organization's internal documents, manuals, or records.
- Cost of retraining: updating a model's internal knowledge through retraining or fine tuning is expensive and slow compared to updating a document store.

RAG addresses these limitations by retrieving relevant, up to date, and domain specific content at query time and supplying it to the model as context, which allows the model to answer questions about material it was never trained on.

Embeddings and Semantic Search

An embedding is a numeric vector representation of a piece of text. Text with similar meaning is mapped to vectors that are close to each other in this vector space, even if the wording is different. This is what allows semantic search to work: instead of matching exact keywords, the system compares the meaning of the query to the meaning of the stored text.

Semantic search is the process of finding the most relevant pieces of text by comparing the embedding of a query against the embeddings of stored documents, and returning the ones that are closest in vector space rather than the ones that share the most words.

Vector Databases

A vector database is a storage system designed specifically to hold embeddings and search through them efficiently. Rather than scanning every stored vector one by one, vector databases use indexing structures that allow them to quickly find the vectors that are most similar to a given query vector, even when the collection contains a large number of documents.

In this project, ChromaDB is used as the vector database. It stores each text chunk together with its embedding and an identifier, and it exposes a query function that returns the chunks whose embeddings are closest to a given query embedding.

Chunking Strategies

Documents are usually too long to embed and retrieve as a single unit, so they are split into smaller pieces called chunks before embedding. Chunking affects retrieval quality directly: chunks that are too large may contain multiple unrelated ideas, which makes the embedding less precise, while chunks that are too small may lose the surrounding context needed to answer a question.

A common strategy, and the one used in this project, is fixed size chunking with overlap. The text is split into segments of a set character length, and each segment overlaps with the previous one by a smaller number of characters. The overlap helps to preserve context that would otherwise be cut off at a chunk boundary.

Embedding Models

An embedding model is the component that converts raw text into its vector representation. These models are trained so that semantically similar sentences produce vectors that are close together. Smaller embedding models are faster and require less memory, while larger models generally capture meaning more precisely at the cost of speed.

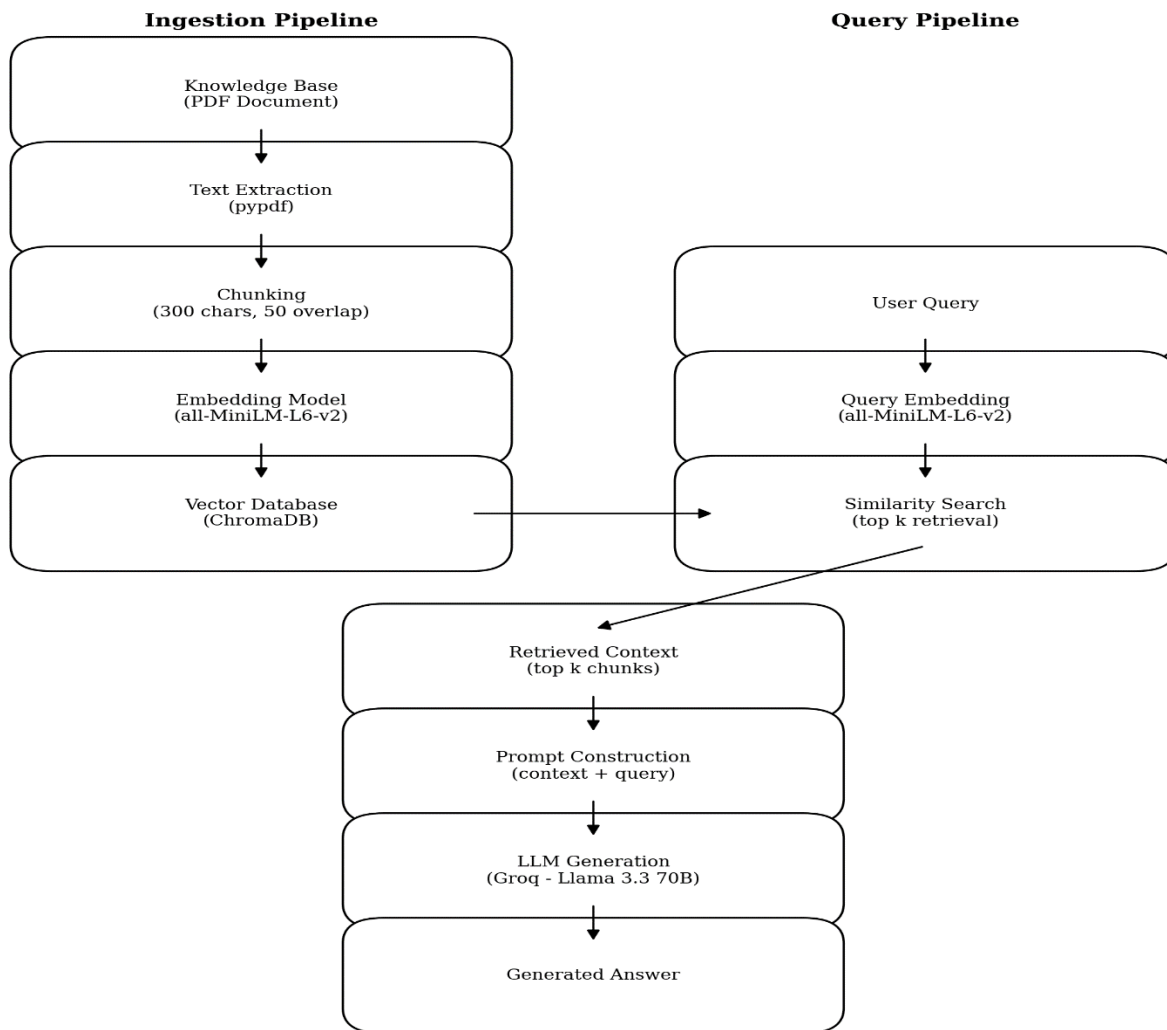
This project uses the all-MiniLM-L6-v2 model from the sentence-transformers library. It is a compact model that produces 384 dimensional embeddings and is commonly used for semantic search tasks because it offers a good balance between speed and retrieval quality.

Similarity Search

Once both the documents and the query are represented as embeddings, similarity search is the step that compares the query embedding against every stored document embedding and ranks them by closeness. The closeness between two vectors is typically measured using a distance or similarity metric such as cosine similarity. The top ranked chunks, referred to as the top k results, are then selected as the context to be passed to the language model.

RAG Pipeline Architecture

The complete pipeline is made up of two connected stages: an ingestion pipeline that prepares the knowledge base, and a query pipeline that answers user questions using that knowledge base. The diagram below shows how the two stages connect.



In the ingestion pipeline, the PDF knowledge base is loaded and its text is extracted, the text is split into overlapping chunks, each chunk is converted into an embedding, and the chunks together with their embeddings are stored in the vector database. In the query pipeline, the user question is converted into an embedding using the same embedding model, and this query embedding is compared against the stored chunk embeddings through similarity search. The retrieved chunks

are combined with the original question into a single prompt, which is passed to the Groq hosted language model. The model then generates an answer based only on the retrieved context.

The table below summarizes each component of the pipeline and the tool used to implement it in the notebook.

Component	Tool Used	Purpose
Document Loading	pypdf	Extracts raw text from the PDF knowledge base
Chunking	Custom function	Splits text into overlapping segments of fixed size
Embedding	sentence-transformers (all-MiniLM-L6-v2)	Converts text chunks into numeric vectors that capture meaning
Storage	ChromaDB	Stores chunk embeddings for fast retrieval
Retrieval	Chroma similarity query	Finds the top k chunks closest to the query embedding
Generation	Groq API (Llama 3.3 70B)	Generates the final answer using the retrieved context

Implementation and Explanation of the Notebook

This section explains what each part of the notebook does and reports the actual outputs produced when the notebook was executed.

Knowledge Base and PDF Creation

A short passage of original text on renewable energy sources was written directly in the notebook as the knowledge base. This text was then written into a PDF file named `knowledge_base.pdf` using the `fpdf` library, so that the pipeline could be demonstrated starting from a PDF document rather than from plain text already held in memory.

Text Extraction

The PDF file was read back using `pypdf`, and the text of every page was concatenated into a single string. When executed, the extracted text had a length of 1890 characters, confirming that the full passage was recovered correctly from the PDF.

Chunking

The extracted text was split using a chunk size of 300 characters with an overlap of 50 characters between consecutive chunks. Running this function on the 1890 character text produced 8 chunks in total.

Embedding Generation

The `all-MiniLM-L6-v2` model was loaded through `sentence-transformers`, and all 8 chunks were encoded into embeddings in a single batch call. Each chunk is represented internally as a 384 dimensional vector after this step.

Vector Database Storage

A ChromaDB client was created and a collection named `knowledge_base` was set up. The 8 chunks, their corresponding embeddings, and a simple numeric identifier for each chunk were added to this collection, making the chunks searchable by vector similarity.

Similarity Search Function

A retrieval function was defined that embeds an incoming query using the same embedding model and then queries the ChromaDB collection for the top k closest chunks, with k set to 3 by default. This function returns the text of the retrieved chunks.

RAG Pipeline Function

A function named `rag_answer` combines retrieval and generation. It calls the retrieval function to obtain the relevant chunks, joins them into a single context string, builds a prompt that presents the context followed by the question and an instruction to answer using only that context, and sends this prompt to the Groq API using the Llama 3.3 70B versatile model with a temperature of 0.3 and a top_p of 0.9.

Test Queries and Results

The pipeline was tested with two questions to confirm that the retrieved context was being used correctly by the model.

Query 1: what is offshore wind energy and why is it more consistent

Answer: Offshore wind energy is generated using turbines in wind farms built offshore. It is more consistent because wind speeds over water are higher and steadier than over land.

Query 2: what factors affect the environmental benefit of biomass energy

Answer: The environmental benefit of biomass energy depends on how sustainably the source material is harvested and replaced.

Both answers match the corresponding information in the original knowledge base passage, which confirms that the retrieval step is returning the correct chunks and that the language model is generating its answer from the retrieved context rather than from general knowledge.

Conclusion

This task implemented a complete retrieval augmented generation pipeline, covering document loading, chunking, embedding, vector storage, similarity search, and grounded answer generation through the Groq API. The test results confirm that the chatbot answers questions using only the information contained in the supplied knowledge base.